

Lab 4 - VibeCoding: Chat History

Name: R Hemanth Reddy | SRN: PES1UG24AM390 | Section: 5-G

Assigned Repo: 28_defender (SETAPESU26) | Tool used: AI coding assistant (Claude)

Project: single-file Defender-lite clone in Pygame. One deliberate bug to fix (radar) and three empty feature functions to implement. Below is my prompting / debugging session.

Getting the game running

YOU (Hemanth):

I cloned my assigned repo **28_defender** and read the README. It's a Pygame "Defender" clone with 1 bug and 3 features left as empty functions. Installed pygame and ran *python game.py* - the game works, ship thrusts left/right, I can shoot landers, humanoids get abducted. Before I change anything I recorded 10s of the current behaviour. Now help me go task by task, I want to actually understand each fix, not just paste code.

AI ASSISTANT:

Good approach. Let's take them one at a time and verify each in-game before moving on. Start with the radar bug since that's the one deliberate defect; the other three are feature functions returning None right now. Tell me what you notice about the radar when you play.

Task 1 - Fix the radar bug

YOU (Hemanth):

So the thin strip at the top (the radar) is supposed to show every humanoid, lander and my ship across the whole world. But in game all the dots just stay bunched in the **middle** of the radar near my ship, even when I fly far away. Humanoids are spread across a world 3200 wide but radar doesn't show that. Here's the function:

```
def draw_radar(self, screen):
    pygame.draw.rect(screen, (10, 10, 30), (0, 0, VIEW_W, RADAR_H))
    pygame.draw.rect(screen, (90, 90, 140), (0, 0, VIEW_W, RADAR_H), 1)
    blips = [(h.x, h.y, (90, 230, 120)) for h in self.humanoids]
    blips += [(l.x, l.y, (255, 90, 90) if l.mutant else (230, 200, 60)) for l in self.landings]
    blips.append((self.player.x, self.player.y, (255, 255, 255)))
    for x, y, color in blips:
        rx = self.screen_x(x) % VIEW_W
        ry = (y - PLAY_TOP) / (VIEW_H - PLAY_TOP) * (RADAR_H - 8) + 4
        pygame.draw.rect(screen, color, (rx - 2, ry - 2, 4, 4))
```

YOU (Hemanth):

I think the problem is this line: `rx = self.screen_x(x) % VIEW_W`. `screen_x` is for the main viewport right? Let me check what it does.

AI ASSISTANT:

Exactly the right instinct. Look at `screen_x`:

```
def screen_x(self, x):
    return VIEW_W / 2 + wrap_delta(self.player.x, x)
```

AI ASSISTANT:

That returns a **screen-space** position - it's *VIEW_W/2 plus the distance from the player*. So every blip is drawn relative to where the player is, which is why they all cluster around the centre of the radar. The radar should instead show **absolute world position**: world `x = 0` maps to the left edge of the

radar, world x = WORLD_W maps to the right edge. So the mapping should be:

```
rx = (x % WORLD_W) / WORLD_W * VIEW_W
```

AI ASSISTANT:

That spreads blips across the full radar width regardless of where the player is. I added % WORLD_W so anything slightly out of range still wraps cleanly.

YOU (Hemanth):

That makes sense - screen space vs world space, the README literally mentions that. Let me swap just that one line and keep ry the same (vertical mapping was fine). Updated:

```
for x, y, color in blips:
    # Map absolute world position (0..WORLD_W) across the full radar width,
    # so the radar shows the whole wrapping world instead of clustering near the player.
    rx = (x % WORLD_W) / WORLD_W * VIEW_W
    ry = (y - PLAY_TOP) / (VIEW_H - PLAY_TOP) * (RADAR_H - 8) + 4
    pygame.draw.rect(screen, color, (rx - 2, ry - 2, 4, 4))
```

YOU (Hemanth):

Ran it - now the blips are spread right across the top strip and the white player dot moves along the radar as I fly. The main viewport still centres on me. Bug fixed. Committing this as its own commit.

AI ASSISTANT:

Nice - that's Task 1 done. The viewport still uses screen_x (correct), only the radar now uses world-space. On to the features.

Task 2 - Implement sky_color(wave)

YOU (Hemanth):

Next the empty sky_color(wave). It's called as screen.fill(sky_color(self.wave) or (5, 5, 20)). README says return an (r,g,b) or None, and idea is to brighten / shift hue as waves go up. I want it to change colour each wave but NOT get so bright that I can't see the sprites.

AI ASSISTANT:

Shifting hue is easiest in HSV then converting to RGB - Python has colorsys in the standard library. Rotate the hue a bit per wave and raise the brightness (value) slightly, but clamp it so it never washes out the sprites:

```
import colorsys # add to the imports at the top

def sky_color(wave):
    hue = (0.62 + 0.11 * (wave - 1)) % 1.0 # start deep blue, rotate per wave
    value = min(0.16 + 0.025 * (wave - 1), 0.45) # brighten slowly, capped at 0.45
    r, g, b = colorsys.hsv_to_rgb(hue, 0.65, value)
    return (int(r * 255), int(g * 255), int(b * 255))
```

YOU (Hemanth):

Why cap value at 0.45? Just so background stays darker than the ship/enemies?

AI ASSISTANT:

Right - the ship is near-white and enemies are bright yellow/red. Keeping the sky's value <= 0.45 (max ~115/255 per channel) guarantees good contrast so gameplay stays readable even at high waves. The (wave - 1) means wave 1 keeps the original deep-blue near-black feel.

YOU (Hemanth):

Added `import colorsys` at the top and pasted it. Tested by surviving to wave 2 and 3 - sky noticeably shifts from blue-black towards warmer tones but sprites are still clear. Committing Task 2.

Task 3 - Implement `on_humanoid_rescued(humanoid)`

YOU (Hemanth):

Now `on_humanoid_rescued(humanoid)`. It's called in `update()` right after the +500 bonus when I catch a falling humanoid. Return value is ignored and it only gets the humanoid - so I can't easily touch the player or score from inside it. README suggests a "+500" popup or an invulnerability boost. A popup feels cleaner and will show up nicely in my after-video. How do I do a floating popup if the function only has the humanoid?

AI ASSISTANT:

Since the return is ignored, the function has to work by side effect. A clean way: keep a module-level list of popups, and have this function append one at the humanoid's world position. Then the game loop ages them and the draw loop renders them. Three small pieces:

```
# 1) module-level, near the top (next to PHASES)
POPUPS = [] # floating "+500" reward popups

# 2) the function itself
def on_humanoid_rescued(humanoid):
    POPUPS.append({"x": humanoid.x, "y": humanoid.y - 18,
                  "text": "+500 RESCUED!", "life": 1.2})
```

YOU (Hemanth):

Ok and then I age them + draw them. I'll make them drift upward and fade out so it looks like a proper pop. Where should I update them - inside `Game.update()`?

AI ASSISTANT:

Yes. Add a short block in `update()` (I put it right after `update_bullets()`), and clear the list in `reset()` so stale popups don't survive a restart:

```
# in Game.reset(), first line:
POPUPS.clear()

# in Game.update(), after self.update_bullets(dt):
for popup in POPUPS:
    popup["y"] -= 30 * dt # drift upward
    popup["life"] -= dt
POPUPS[:] = [p for p in POPUPS if p["life"] > 0]
```

YOU (Hemanth):

And the drawing - I want it in world space so the text appears where the humanoid was, and fade the colour as life runs out. I used `screen_x()` to convert and only draw if it's on screen:

```
# in Game.draw(), just before self.draw_radar(screen):
for popup in POPUPS:
    sx = self.screen_x(popup["x"])
    if -60 < sx < VIEW_W + 60:
        fade = max(0, min(255, int(255 * popup["life"] / 1.2)))
        label = self.font.render(popup["text"], True, (255, 255, fade))
        screen.blit(label, label.get_rect(center=(sx, popup["y"])))
```

YOU (Hemanth):

Tested it: shot a lander carrying a humanoid so it dropped, flew into the falling guy, and "+500 RESCUED!" pops up and floats/fades. Score jumps 500 too. Looks great. Committing Task 3.

AI ASSISTANT:

That's a self-contained reward system that respects the ignored-return constraint - good. Last one.

Task 4 - Implement `bonus_life_threshold()`

YOU (Hemanth):

Last one, `bonus_life_threshold()`. Looking at `update()`, the bookkeeping is already written:

```
threshold = bonus_life_threshold()
if threshold and self.score // threshold > self.bonus_awarded:
    self.bonus_awarded = self.score // threshold
    self.lives += 1
```

YOU (Hemanth):

So I literally just need to return a number and it awards a life each time the score crosses a new multiple of it. README suggests 10000. That's it right?

AI ASSISTANT:

Correct - no other wiring needed. Return 10000 (or None to disable). Each time score passes 10000, 20000, 30000... `self.lives += 1`.

```
def bonus_life_threshold():
    """Extra life each time the score crosses a new multiple of 10000."""
    return 10000
```

YOU (Hemanth):

Done. I verified the logic by temporarily setting my score to 10000 in a quick test - lives went from 3 to 4 exactly once. Reverted the test. Committing Task 4.

Final verification

YOU (Hemanth):

All four done in separate commits. I ran the game end-to-end: radar spreads across the world, sky changes per wave, rescue popup shows, bonus life triggers, and nothing crashes over a long play. Recorded the 10s after-video. Thanks for walking through the why on each one.

AI ASSISTANT:

Summary of what we changed:

1. **Radar** - plot blips by absolute world x instead of player-relative `screen_x`.
2. **sky_color** - HSV hue rotation + capped brightness per wave.
3. **on_humanoid_rescued** - module-level POPUPS list drives a floating "+500 RESCUED!" that drifts and fades.
4. **bonus_life_threshold** - return 10000; existing bookkeeping awards the life.

All four verified in-game. Good work driving the debugging.